Lecture 2

04 October 2026 16:29

1. Use and importance of some IMPORT statements

import torch
import torch.nn as nn
from torch,nn import functional as F

1. Pytorch library

This is the root PyTorch library package.
What functions we can use in it: 1. tensors, 2. math operations, 3. GPU/device control and 4. gradient control

- **Tensors:** It provides torch.Tensor, the primary data structure in PyTorch (similar to NumPy arrays, but capable of running on GPUs).
- **Core Math Operations:** Mathematical operations like matrix multiplication (torch.matmul), concatenation (torch.cat), and reshaping (torch.reshape / .view()).
- **GPU / Device Control:** Moving data between CPU and GPU (torch.device('cuda'), .to('cuda')).
- **Gradient Control:** Managing automatic differentiation context, such as disabling gradient tracking during evaluation (with torch.no_grad():).

2. Import torch.nn as nn

The nn (Neural Network) module provides the object-oriented building blocks for constructing neural network architectures.

- **Base Class (nn.Module):** Every custom model or layer you build inherits from nn.Module.
- **Stateful Layers:** Contains predefined network layers that **store learnable parameters** (**W** weights and **b** biases).

```
python
class BigramLanguageModel(nn.Module):
    def __init__(self, vocab_size):
        super().__init__()
        # Keeps track of trainable embedding weights
        self.token_embedding_table = nn.Embedding(vocab_size, vocab_size)
```

- **Loss Functions (as objects):** nn.CrossEntropyLoss(), nn.MSELoss().

V IMP

1. Base class point: Where nn.Module is like a data block made by the torch lib for the neural networks, so we can use those functional directly so that we don't need to the neural network again.
2. Stateful layers point: we can make learnable parameters form self. ___
3. Loss functions point: when we train the model as it's not always it will give the right answer, it will be near the answer so the loss calculation and other thing can be calculated.

3. From torch.nn import functional as F

F contains **functional** (stateless) equivalents of neural network operations.

Functions in F take inputs, perform a mathematical operation, and return outputs **without storing any state or trainable weights inside the function itself**.

- **Activation Functions:** F.relu(x), F.gelu(x), F.sigmoid(x).
- V IMP **Probability / Softmax:** F.softmax(logits, dim=-1) (takes raw logits and turns them into probabilities).
- **V IMP Loss Functions (as functions):** F.cross_entropy(logits, targets).
- **Data Transformations:** Padding (F.pad), one-hot encoding (F.one_hot).

Summary

Key Comparison: nn vs F

A common question when starting PyTorch is: *"Why do both nn and F have things like ReLU or CrossEntropy?"*

- **Use nn** inside __init__() when you need an **object that holds trainable weights** (like nn.Linear or nn.Embedding).
- **Use F** inside forward() when performing **pure mathematical operations without saved weights** (like F.softmax() or F.cross_entropy()).

i.e. when not to save the weights use ---> F
And when weights need to be saved use ----> nn

Simple example:

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class SimpleMLP(nn.Module):
    def __init__(self, in_features, out_features):
        super().__init__()
        # State: Linear layer holds weight matrix W and bias b
        self.fc = nn.Linear(in_features, out_features)

    def forward(self, x):
        # Apply linear transformation (uses saved weights)
        logits = self.fc(x)

        # Stateless operation: F.softmax doesn't store any weights
        probs = F.softmax(logits, dim=-1)
        return probs
```

1.C.1 what the actual code looks now step by step with understanding

```
import torch
import torch.nn as nn
from torch.nn import functional as F

torch.manual_seed(1337)

class BigramLanguageModel ( nn.Module ):
    def __init__(self, vocab_size):
        super().__init__()
            # each token look for the logits(predictions) for the next token from a look-up table,
            # called as Embedding table
             self.token_embedding_table = nn.Embedding(vocab_size, vocab_size)
```

BigramLanguageModel is a child of the nn.Module
Super().__init__() is used to call the nn.Module __ini__ method

1. BigramLanguageModel is a child of nn.Module

And it will inherit all the features of the nn.Module

2. What's super().__init()

Super() is a super class / parent class and __init__() will help to call the nn.Module's __init__(initialization)

3. What's does __init__(initialization) of the nn.Module conatins under the hood

It contain the stored weights nad biases we will used in the model, contains calculation os the hidden layes like nn.Liner, nn.Embedding, etc and non-trainable tensors like layerNorm, etc.

Part 1: What is super().__init__() and why is it required?

In Python, super() refers to the **parent class** (also called the superclass).
When you write:

```
python
class BigramLanguageModel(nn.Module):
   def __init__(self, vocab_size):
      super().__init__()  # Calls nn.Module's __init__() method
```

You are declaring that BigramLanguageModel is a child of nn.Module. Calling super().__init__() runs PyTorch's setup code inside nn.Module.

What does nn.Module.__init__() secretly do under the hood?
Inside nn.Module, PyTorch creates internal dictionaries to track your model:

1. **_parameters**: A list of all trainable weights and biases in your model.
2. **_modules**: A list of all sub-layers (like nn.Linear, nn.Embedding) assigned to self.
3. **_buffers**: Non-trainable tensors (like attention masks or batch norm running averages).

V IMP
What happens if you forget super().__init__()?

PyTorch will raise an error like AttributeError: cannot assign module before Module.__init__() call. Without it, PyTorch won't know which layers or weights belong to your model!

Part 2: What is nn.Module?

Your understanding is spot on! nn.Module is PyTorch's **master blueprint/wrapper** for neural networks.
Instead of writing everything from scratch (like manually initializing random weight matrices, writing matrix multiplication loops, tracking gradient pointers, writing GPU movement code), inheriting from nn.Module automatically gives your class super-powers:

- **Automatic Parameter Tracking:** Call model.parameters() to get all trainable weights for your optimizer (torch.optim.AdamW(model.parameters(), lr=1e-3)).
- **Hardware Switching:** Call model.to('cuda') to automatically move every layer's weights to the GPU.
- **Saving & Loading:** Call model.state_dict() and torch.save() to save your trained model to disk.
- **Train/Eval Modes:** Call model.train() or model.eval() to toggle behaviors like Dropout or LayerNorm

1.C.2

```
Def forward(self, idx, targets):
    logits = self.token_embedding_table(idx)   # output of shape (B, T, C)

    B, T, C = logits.shape

    # 1. reshape logits (B,T,C) into logits (B*T, C)
    logists = logits.view(B*T, C)            # gives output in 2 dimension (N, C)

     # 2. reshape targets (B,T) into targets (B*T)
    targets = logits.view(B*T)

    loss = F.cross_entropy(logits, targets)
```

Step-by-Step Flow when you call model(idx, targets)

1. **m = BigramLanguageModel(vocab_size)**
   - Calls __init__().
   - Initializes nn.Module tracking via super().__init__().
   - Creates a trainable embedding lookup matrix of size vocab_size x vocab_size.
2. **logits, loss = m(idx, targets)**
   - Notice you call m(idx, targets), **not** m.forward(idx, targets).
   - nn.Module intercepts the () call and automatically invokes the forward() method while handling PyTorch hooks and gradient tracking.
3. **Inside forward():**
   - **Lookup:** Every token integer in idx pulls its corresponding row of logits from token_embedding_table.

return logits, loss

- **Reshape for Loss:** PyTorch's F.cross_entropy function expec a 2D matrix of (batch_size * sequence_length, num_classes) So we stretch/flatten the 3D tensor (B, T, C) into 2D (B*T, C)
- **Loss:** F.cross_entropy internally applies Softmax to logits an computes the negative log-likelihood loss against targets.

What are logits?
There
Are the raw scores obtained from the output before the final layer like activation layes (in LLM, it can be softmax) before converting it into probabilities.

1. Logits in Deep Learning (Neural Networks)

In a classification neural network, logits are the linear outputs of the final layer right **before** an activation function (like **Softmax** or **Sigmoid**) is applied.
Flow in a Classification Model:

[Equation]

- **Logits**: Range from [Equation] . They indicate how strongly the network favors each class relative to the others, but they do not sum to [Equation] .
- **Probabilities**: Range from [Equation] and sum to [Equation] (after Softmax/Sigmoid).

2. Are Logits a Tensor?

Yes, logits are always a PyTorch/TensorFlow Tensor.
For a batch of sequences being processed, the shape of the Logits Tensor is usually:

[Equation]

Example in Python (PyTorch):

python
```
# Batch of 2 sequences, each of length 5 tokens, with a vocab of 10,000 words
logits = model(input_ids)
print(type(logits))
# Output: <class 'torch.Tensor'>
print(logits.shape)
# Output: torch.Size([2, 5, 10000])
```

V IMP

3. How do Logits help in Training?

Logits are the central component used to compute the **Loss** during training:

1. **Compare Logits to Actual Next Token**: Suppose the model received the prompt "The cat sat on the " and the target next token in the training text is "mat" (which has Token ID 87).
2. **Calculate Loss (Cross-Entropy Loss)**: The loss function (e.g., torch.nn.CrossEntropyLoss) takes the entire **Logits Tensor** (all 50,000 scores) and the ground truth index 87: [Equation]
   - If logit[87] is very high compared to other logits → **Loss is small**.
   - If logit[87] is low compared to other logits → **Loss is large**.
3. **Backpropagation**: The calculated loss sends gradients backward through the network, updating the weights (including the embedding table and attention layers) so that next time, the model gives a higher logit score to "mat".

Summary

- **Embedding Table**: Input lookup → converts ID 24 into a feature vector.
- **Logits**: Output of the model [Equation] **a Tensor** containing unnormalized prediction scores for all tokens in the vocabulary.
- **Training Role**: Passed into Cross-Entropy loss to measure how wrong the predictions are and guide weight updates via backpropagation.

**How is each word (given as input) converted and reffered to the embedding table,
And what's the IMPORTANCE of the EMBEDDING TABLE for each token**

1. **So basically, each alphabet is given some token id (like a = 0, b = 1, c = 4, … etc)**
2. **This token id's are nothing but the vocab size we mde earlier,**
3. **We have taken all the characters included in the text(65) and converted them in a numbers using stoi (string to integer) and back into words or letters for getting output and for the model to learn as itos (integer to string)**
4. **So when the input like for example hello is given as input**
5. **It is converted into token id's like this**

If the user inputs the text "hello":

python
```
text = "hello"
token_ids = [stoi[c] for c in text]
# Result: [12, 5, 15, 15, 18]
```
Now our text "hello" is represented as PyTorch tensor torch.tensor([12, 5, 15, 15, 18]).

6. **V IMP**

So the embedding table is made of size vocab size cross embedding dim
Here, the embedding dimension is nothing but the features each alphabet(a, b, c, ..) possessed from referring to other.

V IMP
At first the numbers In the row are totally random and later using back-propogation, the words are changed (updated) by training, and at last they are arranged again in order to which word is closer to the input word, or alphabet.
----------this is done using neural network----------

V IMP SUMMARY OF above

Part 1: How is input text given different Token IDs?

Before touching PyTorch, we create a process called **Tokenization**.

1. **Build a Vocabulary (vocab):** We scan the entire dataset (e.g., Shakespeare text) and find all unique characters. Suppose there are 65 unique characters: ['\n', ' ', '!', '$', ..., 'a', 'b', ..., 'z'].
2. **Create Lookup Dictionaries:**
   - **stoi** (string to integer): maps each character to a number. {'a': 0, 'b': 1, 'c': 2, ..., 'h': 12, 'e': 5, 'l': 15, 'o': 18}
   - **itos** (integer to string): maps each number back to its character.

Example:
If the user inputs the text "hello":

```
python
text = "hello"
token_ids = [stoi[c] for c in text]
# Result: [12, 5, 15, 15, 18]
```

Now our text "hello" is represented as PyTorch tensor torch.tensor([12, 5, 15, 15, 18])

Part 2: How is the Embedding Table made? (Rows & Columns)

When you write in PyTorch:

```
python
embedding_table = nn.Embedding(vocab_size=65, embedding_dim=4)
```

1. **Rows = vocab_size (65):** One row for every character in our vocabulary.
2. **Columns = embedding_dim (4):** The number of features representing each character.
3. **Initial Values:** When PyTorch creates this table, **it fills all cell values with random small numbers** (e.g., -0.43, 0.12, 0.89).

```
Embedding Table Matrix (65 rows x 4 columns)
       Col 0    Col 1    Col 2    Col 3
Row 0   [ -0.12,   0.84,   0.05,  -0.42 ]  ('\n')
Row 1   [  0.45,  -0.11,   0.92,   0.31 ]  ('a')
...
Row 12  [  0.18,  -0.75,   0.41,   0.90 ]  ('h')  <-- Filled with random numbers initially
...
Row 64  [ -0.89,   0.62,  -0.34,   0.15 ]  ('z')
```

MOST IMP THING TO UNDERSTAND

How do columns get meaningful values?

1. At the start, the numbers are **completely random**.
   (this is done using the trainable weight by pytorch)

2. During training, the model makes predictions, calculates the loss, and runs loss.backward().

(the meaning of loss is from the neural network,
Let's say: when one prediction is done or made, we cannot tell this is the right one or optimal one need for the output to the user, so, thing we do is, we calculate the loss from hidden layer, and travel backward through the layers.)

V IMP key points to understand

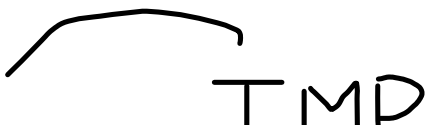
3. PyTorch uses backpropagation to **adjust these random numbers** in the table after every batch.

(back-propogation is the process in ML learning were we take back the step once we know the path to the solution is taking longer step although if it reaches the output we need, but we need your model to be good and be quick i.e. it should take shorter path even if it knew half the way before, and should take that second most optimal path/ solution.)

V IMP how the embedding table is important

4. Over time, characters that behave similarly in text get pushed toward similar numbers in their rows!

(just like the king and queen example, as there are high chances that these two are use or can come together, this takes place using the help of the embedding table)



Part 3: How do Token IDs look up rows in the Embedding Table?

When we pass our token IDs [12, 5, 15, 15, 18] into Embedding_table:

```
python
input_ids = torch.tensor([12, 5, 15, 15, 18])  # "hello"
embedded_output = embedding_table(input_ids)
```

PyTorch performs a **fast row lookup**:

- ID 12 [Equation] copies Row #12 ([0.18, -0.75, 0.41, 0.90])
- ID 5 [Equation] copies Row #5
- ID 15 [Equation] copies Row #15
- ID 15 [Equation] copies Row #15
- ID 18 [Equation] copies Row #18

Output shape becomes (5 tokens, 4 embedding_dim).

Part 4: What happens next? What does it generate later?

This depends on whether you are using the **Simple Bigram Model** or a **Full GPT Model**:

Scenario A: Simple Bigram Model (embedding_dim = 65)

In the simple Bigram model, we set embedding_dim = vocab_size = 65.

1. When we look up row #12 for 'h', the row already contains **65 numbers**.
2. **V imp : These 65 numbers are directly the Logits (predictions)** for what character comes after 'h'.
3. We pass these 65 logits into F.softmax() to get probabilities (e.g., 80% chance of 'e', 5% chance of 'o').
4. V IMP : We sample from those probabilities to pick the next character!

Extra information for bigger GPT- models

Scenario B: Full GPT Model (embedding_dim = 384 or 768)

In GPT, the embedding gives us a vector of length 384 (or 768).

1. **Pass through Transformer Blocks:** That 384-dimensional vector passes through multiple **Self-Attention** and **Feed-Forward (MLP)** layers. Here, tokens look at previous tokens and combine context (e.g., 'h' + 'e' + 'l' + 'l' together).
2. **Final Linear Layer (Head):** At the very end of the network, a final nn.Linear(384, 65) projects the 384-dimensional vector back into **65 Logits**.
3. **Generation:**
   - Softmax turns those 65 logits into probabilities.
   - PyTorch picks/samples the next character (e.g., 'o').
   - We append 'o' to our text, and repeat the process to generate the next letter!

Summary Diagram of Generation:

[Equation] [Equation] [Equation] [Equation] ->" [Equation]

Slightly real meaning of the back propogation

The Tiny Tweak: What Backpropagation Actually Is

In your notes, you described backpropagation as taking a step back to find a "shorter/optimal path" (like backtracking in a maze).
In Neural Networks, **Backpropagation** works slightly differently:

1. The Loss (How wrong the model is):
The loss is a single number (e.g., 4.32). It tells us: *"Given the input 'h', the model assigned a low probability to the correct target letter 'e'."*

2. Backpropagation (loss.backward()):
Backpropagation works backward from the loss output through all the layers down to the Embedding Table to calculate **gradients** (slopes).

- A gradient answers one simple question for every single number in the table:
  "If I increase this number slightly, will the loss go UP or DOWN?"

3. Weight Update (The Optimizer):
The optimizer (like AdamW) takes a small step in the direction that **lowers the loss**.

- If increasing a number makes the model better at predicting 'e' after 'h', that number is increased.
- Over thousands of iterations, this nudges words/characters that appear in similar contexts (like king and queen) to have similar row values!

Step-by-Step Breakdown of the Forward Pass

1. The Inputs: idx and targets

1. **idx (Input Tokens):** A 2D Tensor of shape **(B, T)** containing current prompt token IDs.
   - *Example:* [Equation] sequences, [Equation] characters each [Equation] a [Equation] grid of numbers.
2. **targets (Correct Next Tokens):** A 2D Tensor of shape **(B, T)** containing what the next character *should* be for every position in idx.
3. **Why is targets=None optional?**
   - **During Training:** We pass both idx and targets so PyTorch can calculate loss.
   - **During Generation (Inference):** We only pass idx (we don't have targets because we are asking the model to invent new text!).

V IMP ----------SEE HOW THE dimension are need----------

2. The Lookup: logits = self.token_embedding_table(idx)

- Input idx shape: **(B, T)** (e.g., 4 x 8).
- Output logits shape: **(B, T, C)** (e.g., 4 x 8 x 65).

What just happened? For every single integer in that [Equation] grid of tokens, PyTorch looks up its 65-dimensional row in token_embedding_table. Now we have 65 unnormalized prediction scores (logits) for every single token position!

V.IMP Explaination

How to visualize a 3D Tensor (B, T, C) like (4, 8, 65)?

Think of a 2D matrix vs a 3D tensor like a **flat sheet of paper vs a stack of papers**:

1. Input idx (2D Matrix of Shape 4 x 8):

It is a simple 2D grid with 4 rows (batch size [Equation]) and 8 columns (sequence length [Equation]). Inside each cell is **1 single integer** (a token ID):

Seq 1 : [12, 5, 15, 15, 18, 1, 20, 4] it's a batch of matrix (4 * 8)
Seq 2 : [2, 19, 8, 1, 5, 12, 12, 18]
Seq 3 : [15, 18, 20, 1, 4, 8, 5, 19]
Seq 4 : [1, 5, 12, 12, 18, 2, 19, 8]

V IMP:
Total cells = [Equation] integer token IDs.

2. Output logits (3D Tensor of Shape 4 x 8 x 65): most IMP

Every single integer in that [Equation] grid gets replaced by a **1D vector of 65 floating-point numbers** (its embedding/logits).

You can visualize this in two easy ways:

- **View 1 (Grid of Vectors):** Keep the same [Equation] grid, but now inside **each cell** sits a list of 65 numbers:

  Seq 1 : [65 numbers], [65 numbers], [65 numbers], ...]
  Seq 2 : [65 numbers], [65 numbers], [65 numbers], ...]
  Seq 3 : [65 numbers], [65 numbers], [65 numbers], ...]
  Seq 4 : [65 numbers], [65 numbers], [65 numbers], ...]

  View 2 (Stack of Matrices): It is simply **4 separate 2D matrices**, each of size [Equation] , stacked on top of each other!

Doubt 2: Does each token get its OWN 65x65 table?

NO! There is only ONE SINGLE master embedding table for the whole model!
The embedding table nn.Embedding(65, 65) is a single shared matrix of size 65 rows x 65 columns.

MASTER EMBEDDING TABLE (65 x 65)
Col 0 Col 1 ... Col 64
Row 0 [0.12, -0.45, ... 0.89] ('\n')
Row 1 [-0.91, 0.33, ... -0.12] ('a')
Row 2 [0.05, 0.11, ... 0.42] ('b')
...
Row 12 [0.44, 1.21, ... -0.88] ('h')
...
Row 64 [-0.89, 0.62, ... 0.15] ('z')

Exact meaning

What happens during lookup:

Every token in your [Equation] batch goes to **this exact same master table** and copies **one row**:

- If token ID 12 ('h') appears at sequence 1 position 0, it pulls **Row #12** (which is a 1D vector of 65 numbers).

- If token ID 5 ('e') appears at sequence 1 position 1, it pulls **Row #5** (a 1D vector of 65 numbers).
- If token ID 12 ('h') appears AGAIN at sequence 2 position 5, it pulls **the exact same Row #12** again!

So for 32 tokens in your batch, you pull 32 rows out of the single master table.
Arranged into batch shape, those 32 rows form your **(4, 8, 65)** logits tensor!

3. Why do we check if targets is None:?

python
```
if targets is None:
    loss = None
```

When generating text, we don't have ground-truth targets. So the model skips loss calculation, returns logits, None, and saves computation time.

4. The Reshape Trick: logits.view(B*T, C)

When targets ARE provided, we enter the else: block:

python
```
B, T, C = logits.shape
logits = logits.view(B * T, C)          # (4, 8, 65) -> (32, 65)
targets = targets.view(B * T)           # (4, 8)     -> (32)
```

Why must we reshape with .view()? PyTorch's F.cross_entropy(input, target) has a strict requirement:

- input MUST be a **2D Tensor** of shape (N, C) [Equation] (total_predictions, num_classes).
- target MUST be a **1D Tensor** of shape (N) [Equation] (total_predictions).

VIMP

In our batch:

- We have [Equation] batches and [Equation] sequence steps.
- That means there are [Equation] total individual character predictions to check!
- .view(B*T, C) stacks all 32 token positions into 32 rows, each with 65 class logits.

Last but not the least

5. Loss Calculation: loss = F.cross_entropy(logits, targets)

F.cross_entropy performs two steps under the hood:

1. Takes the 65 logits for each of the 32 tokens and applies **Softmax** to get probabilities ([Equation] to [Equation]).
2. Compares those probabilities against the actual correct target token ID using **Negative Log-Likelihood**.

If the model gave a high probability to the right target letter, loss will be small (e.g., 0.2). If it guessed wrong, loss will be high (e.g., 4.5).

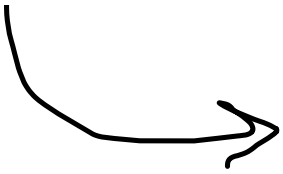
If the prediction is right then probability will be more(near 1) and loss will be low
Similarly, probability will be less (near 0) and loss will be high

Our model should generate less loss

Visual Summary of Tensor Shapes Through Forward Pass:

[Equation]

Step 1 step 2 step 3 final step
V V IMP



Summarizes the whole code process under the hood

It's a V IMP diagram for learning